In listing 10.2 we define a class that is the relational module. It is a single, but complex, neural network that includes an initial set of convolutional layers followed by the key, query, and value matrix multiplications.

Listing 10.2. Relational module

```
class RelationalModule(torch.nn.Module):
    def __init__(self):
        super(RelationalModule, self).__init__()
        self.ch_in = 1
        self.conv1_ch = 16
1
        self.conv2_ch = 20
        self.conv3_ch = 24
        self.conv4_ch = 30
        self.H = 28
2
        self.W = 28
        self.node_size = 36
3
        self.lin_hid = 100
        self.out_dim = 10
        self.sp_coord_dim = 2
        self.N = int(16**2)
4

        self.conv1 =
nn.Conv2d(self.ch_in,self.conv1_ch,kernel_size=(4,4))
        self.conv2 =
nn.Conv2d(self.conv1_ch,self.conv2_ch,kernel_size=(4,4))
        self.conv3 =
nn.Conv2d(self.conv2_ch,self.conv3_ch,kernel_size=(4,4))
        self.conv4 =
nn.Conv2d(self.conv3_ch,self.conv4_ch,kernel_size=(4,4))

        self.proj_shape =
(self.conv4_ch+self.sp_coord_dim,self.node_size)  5
        self.k_proj = nn.Linear(*self.proj_shape)
        self.q_proj = nn.Linear(*self.proj_shape)
        self.v_proj = nn.Linear(*self.proj_shape)

        self.norm_shape = (self.N,self.node_size)
        self.k_norm = nn.LayerNorm(self.norm_shape,
elementwise_affine=True)6
        self.q_norm = nn.LayerNorm(self.norm_shape,
elementwise_affine=True)
        self.v_norm = nn.LayerNorm(self.norm_shape,
elementwise_affine=True)
```